

# ■ WORD SEARCH JOURNEY

*Flutter Clone — Complete Cursor AI App Prompt*

★★★★★ 4.8 Rating | 100M+ Downloads | Google Play

■■ Clean Architecture

■ BLoC Pattern

■ Pixel-Perfect UI

■ Full Monetization

Prepared May 2026 | Full Technical Blueprint for Cursor AI Code Generation

# TABLE OF CONTENTS

|    |                                       |    |
|----|---------------------------------------|----|
| 1  | CURSOR AI — MASTER APP PROMPT         | 3  |
| 2  | APP OVERVIEW & PLAY STORE SCREENSHOTS | 6  |
| 3  | COMPLETE ARCHITECTURE BLUEPRINT       | 9  |
| 4  | APP NAVIGATION FLOW DIAGRAM           | 12 |
| 5  | COMPLETE WIREFRAMES — ALL SCREENS     | 14 |
| 6  | COLOR SYSTEM & DESIGN TOKENS          | 22 |
| 7  | COMPLETE FOLDER STRUCTURE             | 24 |
| 8  | PUBSPEC.YAML — ALL DEPENDENCIES       | 26 |
| 9  | CORE GAME ALGORITHMS (FULL CODE)      | 28 |
| 10 | ALL SCREENS — FULL FLUTTER CODE       | 34 |
| 11 | BLOC STATE MANAGEMENT — FULL CODE     | 58 |
| 12 | DATA LAYER — DATABASE & MODELS        | 66 |
| 13 | MONETIZATION — ADMOB + IAP CODE       | 72 |
| 14 | ADDITIONAL FEATURES CODE              | 78 |
| 15 | DEVELOPMENT ROADMAP & CHECKLIST       | 82 |

# 1. CURSOR AI — MASTER APP PROMPT

Yeh section cursor.ai ko directly paste kiya ja sakta hai. Yeh ek complete app prompt hai jo puri app generate karta hai.

## ■ CURSOR AI PROMPT — COPY & PASTE THIS ENTIRE SECTION

Model: Claude Sonnet 4 (recommended) | Mode: Agent | Context: Max  
Paste the entire prompt below into Cursor AI chat to generate the complete app.

## Master Prompt (Part 1) — Project Setup & Architecture

```
You are an expert Flutter developer. Build a complete, production-ready Flutter app called "Word Search Journey" — a clone of the Google Play game by Bluetile Games (com.playvalve.wsjourney, 100M+ downloads, 4.8 stars).
```

```
== TECH STACK ==
```

```
Flutter SDK: 3.22+ (Dart 3.4+)
State Management: flutter_bloc ^8.1.6 (MANDATORY — no other state management)
Database: drift ^2.18.0 (TypeSafe SQLite)
Ads: google_mobile_ads ^5.1.0
IAP: in_app_purchase ^3.2.0
Animation: lottie ^3.1.2, confetti ^0.7.0
Audio: audioplayers ^6.0.0
DI: get_it ^7.7.0, injectable ^2.4.0
Firebase: firebase_core, firebase_analytics, firebase_crashlytics
Other: shimmer ^3.0.0, animated_text_kit ^4.2.2, intl ^0.19.0
```

```
== ARCHITECTURE ==
```

```
Use Clean Architecture with feature-first folder structure:
- Presentation Layer: Screens, Widgets, BLoC/Cubit
- Domain Layer: Use Cases, Entities, Repository Interfaces
- Data Layer: Repositories, DataSources (Local SQLite + JSON assets)
- Core Layer: Constants, Utils, DI, Error Handling
```

```
== FOLDER STRUCTURE ==
```

```
word_search_game/
  ■■■ lib/
  ■ ■■■ main.dart
  ■ ■■■ app.dart
  ■ ■■■ injection.dart
  ■ ■■■ core/
  ■ ■■■ constants/
  ■ ■ ■■■ app_colors.dart
  ■ ■ ■■■ app_text_styles.dart
  ■ ■ ■■■ app_sizes.dart
  ■ ■ ■■■ game_config.dart
  ■ ■ ■■■ utils/
  ■ ■ ■■■ word_placer.dart
  ■ ■ ■■■ grid_generator.dart
  ■ ■ ■■■ extensions.dart
  ■ ■ ■■■ widgets/
  ■ ■ ■■■ gradient_button.dart
  ■ ■ ■■■ loading_overlay.dart
  ■ ■ ■■■ services/
  ■ ■ ■■■ audio_service.dart
  ■ ■ ■■■ ad_service.dart
  ■ ■ ■■■ analytics_service.dart
  ■ ■■■ features/
```

```

■ ■ ■ ■ splash/
■ ■ ■ ■ home/
■ ■ ■ ■ level_select/
■ ■ ■ ■ game/
■ ■ ■ ■ data/models/
■ ■ ■ ■ domain/
■ ■ ■ ■ presentation/
■ ■ ■ ■ bloc/
■ ■ ■ ■ screens/
■ ■ ■ ■ widgets/
■ ■ ■ ■ profile/
■ ■ ■ ■ shop/
■ ■ ■ ■ settings/
■ ■ ■ ■ data/
■ ■ ■ ■ local/
■ ■ ■ ■ words/
■ ■ ■ ■ assets/
■ ■ ■ data/levels_pack_1.json
■ ■ ■ data/words_dictionary.json

```

## Master Prompt (Part 2) — UI Design Requirements

```

== COLOR SCHEME (EXACT HEX VALUES) ==
Primary Blue: #1565C0 (buttons, headers, active states)
Ocean Blue: #0288D1 (grid border, accents, secondary brand)
Dark Navy: #01579B (deep UI elements)
Success Green: #2E7D32 (found words, stars, completed levels)
Warning Orange: #E65100 (hard levels, hints low alert)
Gold/Coin: #FFA000 (coins, rewards, VIP badge)
Light BG: #E3F2FD (screen backgrounds, relaxing blue-tint)
Grid Cell BG: #FFFFFF (default cell background)
Selected Cell: #B3E5FC (active drag highlight, soft blue)
Found Word: #A5D6A7 (correct word overlay, satisfying green)
Wrong Select: #EF9A9A (wrong selection, brief red flash)
Revealed Word: #CE93D8 (purple for revealed/cheated cells)
Hint Yellow: #FFF59D (hint highlight cells)
Dark Text: #212121 (body text, high contrast)

== TYPOGRAPHY ==
Game Title: Montserrat Black, 36sp, White
Level Name: Poppins SemiBold, 22sp, Dark
Grid Letters: Roboto Mono Bold, 18-22sp (dynamic by grid size)
Word List: Poppins Regular, 16sp, Dark
Found Word: Poppins Regular, 16sp, Green, Strikethrough
Button Text: Poppins SemiBold, 16sp, White
Coins/Score: Orbitron Bold, 20sp, Gold (#FFA000)
Timer: Orbitron Regular, 24sp, White (Red when < 30s)

== GRID CELL STATES (AnimatedContainer) ==
Default: bg=#FFFFFF, border=1px #CCCCC
Hint: bg=#FFF59D, border=2px #F57F17, pulse animation
Being Selected: bg=#B3E5FC, border=2px #0288D1, scale(1.05)
Found Word: bg=#A5D6A7, border=2px #388E3C, shimmer animation
Wrong Selection: bg=#EF9A9A, border=2px #C62828, shake animation
Revealed: bg=#CE93D8, border=2px #7B1FA2, fadeIn effect

== SCREEN REQUIREMENTS ==
1. SPLASH: Travel bg + Lottie logo, 2.5s auto-navigate → /home
2. HOME: Animated world map, destination card, PLAY button, progress bar
3. LEVEL SELECT: Difficulty tabs (Easy/Medium/Hard/Pro), star ratings, hero transition
4. GAME SCREEN (MOST CRITICAL):

```

```

- Top bar: Back | Theme | Stars | Timer | Settings
- Coin row: Coins | Hints | Pause
- Word chips: horizontal scroll, found=green+strikethrough
- Grid: 8x8(Easy) 10x10(Med) 12x12(Hard) 14x14(Pro)
- CustomPainter + GestureDetector for drag selection
- Toolbar: Rotate(FREE) | Hint(15 coins) | Reveal(30) | Shuffle(25)
5. LEVEL COMPLETE: Confetti, star animation, coins earned, rewarded ad offer
6. SHOP: Coin packs, Remove Ads, VIP subscription

```

## Master Prompt (Part 3) — BLoC Events & States

```

== BLOC ARCHITECTURE ==

// GameEvent
abstract class GameEvent extends Equatable {}
class LoadLevel extends GameEvent { final int levelId; }
class CellDragStarted extends GameEvent { final int row, col; }
class CellDragUpdated extends GameEvent { final int row, col; }
class CellDragEnded extends GameEvent {}
class HintRequested extends GameEvent {}
class RevealRequested extends GameEvent {}
class BoardRotated extends GameEvent {}
class ShuffleRequested extends GameEvent {}
class GamePaused extends GameEvent {}
class GameResumed extends GameEvent {}
class ExtraTimeAdded extends GameEvent {}

// GameState
class GameInProgress extends GameState {
  final List<String> grid;
  final List<String> wordsToFind;
  final List<String> foundWords;
  final List<List<int>> selectedCells;
  final int coins;
  final int hintsLeft;
  final Duration elapsed;
  final bool isPaused;
  final bool isCompleted;
  final DifficultyLevel difficulty;
  final SelectionState selectionState;
}
class GameCompleted extends GameState {
  final int stars;
  final int coinsEarned;
  final Duration time;
  final int hintsUsed;
}

// Navigation Routes
/ → SplashScreen
/home → HomeScreen
/levels → LevelSelectScreen (args: themeId)
/game → GameScreen (args: levelId)
/shop → ShopScreen
/profile → ProfileScreen
/settings → SettingsScreen

// Transitions
Splash → Home: FadeTransition
Home → Levels: SlideTransition (right)
Levels → Game: Hero + FadeTransition

```

```
Game → Complete: ScaleTransition overlay
```

## 2. APP OVERVIEW & PLAY STORE SCREENSHOTS

### ■ Game Stats

Developer: Bluetile Games (Barcelona, Spain)  
Downloads: 100 Million+ | Rating: 4.8/5.0 (713K+ Reviews)  
Rank: #4 Top Free Word Game on Google Play  
Package: com.playvalve.wsjourney | Last Updated: March 4, 2026  
Platform: Android + Windows | Offline: Yes | Content: 3+ (All Ages)  
Monetization: AdMob Ads + In-App Purchases (Hybrid Model)

### Official Play Store Screenshots

These screenshots are from the official Word Search Journey Play Store listing. The Flutter clone should replicate this exact design:

|     |                                                                                                                        |
|-----|------------------------------------------------------------------------------------------------------------------------|
| SS1 | <b>Main Home / World Map Screen</b><br>Travel destination pins on animated map, PLAY button, coin counter, daily bonus |
| SS2 | <b>Gameplay Screen</b><br>Active letter grid with swipe highlight, word list on right, tool buttons at bottom          |
| SS3 | <b>Level Selection</b><br>Grid of numbered level cards with star ratings, locked/unlocked states                       |
| SS4 | <b>Theme Destination Selection</b><br>Travel postcard style cards with city images, level counts                       |
| SS5 | <b>Word Found Animation</b><br>Green shimmer overlay on correct word, confetti burst, score popup                      |
| SS6 | <b>Level Complete Screen</b><br>Star rating animation, coins earned, Next Level button                                 |
| SS7 | <b>Shop / IAP Store</b><br>Coin packages, Remove Ads button, VIP pass subscription                                     |
| SS8 | <b>Daily Challenge / Bonus Screen</b><br>Special themed puzzle with streak counter                                     |

### Key UX Insights from 713K+ Reviews

| Feature         | Review Insight & Implementation Guide                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------|
| Rotate Board    | MOST LOVED — "Whoever made rotate board free gets 12 raises" (Nevae Russell, 2026). MUST stay FREE always. |
| Ad Frequency    | "The ads? What ads??" — Minimal & skippable. Show interstitial every 3rd level only, 5s skip.              |
| Hint Placement  | "I accidentally tap hints while looking for words" — Place hint toolbar BELOW grid, not overlapping.       |
| Currency System | "Currency isn't a problem, you can get paid for breathing" — Be generous with coin rewards.                |
| Grid Difficulty | "Levels are too easy, give me 12x12 or 15x15" — Add harder grid size options for Pro difficulty.           |

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| Offline Support | No WiFi required — All 100+ levels bundled in APK assets. No server dependency at launch. |
|-----------------|-------------------------------------------------------------------------------------------|

### 3. COMPLETE ARCHITECTURE BLUEPRINT

### 3.1 Clean Architecture Layers

| ■■ PRESENTATION                                                                                                      | ■■ DOMAIN                                                                                                                        | ■ DATA                                                                                                                                         | ■ CORE                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Screens</li><li>• Widgets</li><li>• BLoC/Cubit</li><li>• Navigator</li></ul> | <ul style="list-style-type: none"><li>• Use Cases</li><li>• Entities</li><li>• Repo Interfaces</li><li>• Domain Models</li></ul> | <ul style="list-style-type: none"><li>• Repositories</li><li>• Local DataSource</li><li>• Remote DataSource</li><li>• Drift Database</li></ul> | <ul style="list-style-type: none"><li>• app_colors</li><li>• app_sizes</li><li>• DI (GetIt)</li><li>• Error Handling</li></ul> |

### 3.2 BLoC State Flow Diagram

```

##### BLoC FLOW – GameBloc #####
■ USER ACTION BLOC EVENT STATE CHANGE■
■ #####
■ Tap level card → LoadLevel(id) → GameLoading■
■ Level loaded → (internal) → GameInProgress■
■ Finger down → CellDragStarted → selectedCells=[cell]■
■ Finger moves → CellDragUpdated → selectedCells=[line]■
■ Finger up → CellDragEnded → validate word■
■ ■■ Valid word → WordFound → foundWords += word■
■ ■ → (if all found) → GameCompleted■
■ ■■ Invalid → WrongSelection → flash red, reset■
■ Hint button → HintRequested → hintsLeft--, flash■
■ Rotate button → BoardRotated → grid = rotate(grid)■
■ Pause button → GamePaused → isPaused = true■
■ All words found → (auto-trigger) → GameCompleted(stars)■
■ #####

```

### 3.3 Dependency Injection (GetIt)

```
// injection.dart
@InjectableViewInit()
void configureDependencies() => getIt.init();

@Module
abstract class AppModule {
  @Singleton AppDatabase get database => AppDatabase();
  @Singleton SharedPreferences get prefs; // async, use @preResolve
  @LazySingleton AdService get adService => AdService();
  @LazySingleton AudioService get audioService => AudioService();
  @LazySingleton AnalyticsService get analyticsService => AnalyticsService();
}

// Feature-level registrations (via @injectable on classes):
@InjectableView class WordPlacerUseCase { ... }
@InjectableView class LoadLevelUseCase { ... }
@InjectableView class SaveProgressUseCase { ... }
@InjectableView class GetCoinsUseCase { ... }
```

[illegible][illegible]

[illegible]



## 5.4 Main Game Screen (Most Critical)

## 5.5 Level Complete Overlay

Page 13 | Clean Architecture • BLoC • AdMob + IAP

## 5.6 Shop Screen

Page 14 | Clean Architecture • BLoC • AdMob + IAP

## 6. COLOR SYSTEM & DESIGN TOKENS

### 6.1 Complete Color Palette

| Element        | Hex Code | Usage                           | Notes               |
|----------------|----------|---------------------------------|---------------------|
| Primary Blue   | #1565C0  | Buttons, headers, active states | Main brand color    |
| Ocean Blue     | #0288D1  | Grid border, secondary          | Secondary brand     |
| Dark Navy      | #01579B  | Deep UI elements                | Depth/shadows       |
| Light Blue BG  | #E3F2FD  | Screen backgrounds              | Relaxing feel       |
| Success Green  | #2E7D32  | Found words, stars              | Positive feedback   |
| Light Green    | #A5D6A7  | Found word cell overlay         | Satisfying green    |
| Warning Orange | #E65100  | Hard level, hints low           | Alert color         |
| Gold/Coin      | #FFA000  | Coins, rewards, VIP             | Premium/currency    |
| White Cell     |          | Default grid cell               | Clean look          |
| Selected Blue  | #B3E5FC  | Active drag cells               | Soft highlight      |
| Wrong Red      | #EF9A9A  | Wrong selection flash           | Brief feedback      |
| Reveal Purple  | #CE93D8  | Cheated/revealed cells          | Distinct from found |
| Hint Yellow    | #FFF59D  | Hint highlight cells            | Eye-catching        |
| Dark Text      | #212121  | Body text                       | High contrast       |
| Grid BG Alt    | #F0F4F8  | Alternate cell rows             | Subtle variation    |

### 6.2 app\_colors.dart — Full Code

```
// lib/core/constants/app_colors.dart
import 'package:flutter/material.dart';

class AppColors {
  // Brand Colors
  static const Color primaryBlue = Color(0xFF1565C0);
  static const Color oceanBlue = Color(0xFF0288D1);
  static const Color darkNavy = Color(0xFF01579B);
  static const Color lightBlueBg = Color(0xFFE3F2FD);

  // Grid Cell States
  static const Color cellDefault = Color(0xFFFFFFFF);
  static const Color cellAlt = Color(0xFFF0F4F8);
  static const Color cellSelected = Color(0xFFB3E5FC);
  static const Color cellFound = Color(0xFFA5D6A7);
  static const Color cellWrong = Color(0xFFEF9A9A);
  static const Color cellRevealed = Color(0xFFCE93D8);
  static const Color cellHint = Color(0xFFFFF59D);

  // Semantic Colors
  static const Color success = Color(0xFF2E7D32);
  static const Color warning = Color(0xFFE65100);
  static const Color gold = Color(0xFFFFA000);
  static const Color darkText = Color(0xFF212121);
}
```

```
// Difficulty Colors
static const Color easy = Color(0xFF4CAF50);
static const Color medium = Color(0xFF2196F3);
static const Color hard = Color(0xFFFF9800);
static const Color pro = Color(0xFFFF4336);

// Gradient
static const LinearGradient primaryGradient = LinearGradient(
  colors: [primaryBlue, oceanBlue],
  begin: Alignment.topLeft, end: Alignment.bottomRight,
);
}
```

## 7. COMPLETE FOLDER STRUCTURE

```
word_search_game/
  ■■■ lib/
  ■ ■■■ main.dart # Entry point
  ■ ■■■ app.dart # MaterialApp + GoRouter
  ■ ■■■ injection.dart # GetIt DI configuration
  ■ ■
  ■ ■■■ core/
  ■ ■ ■■■ constants/
  ■ ■ ■■■ app_colors.dart
  ■ ■ ■■■ app_text_styles.dart
  ■ ■ ■■■ app_sizes.dart
  ■ ■ ■■■ game_config.dart
  ■ ■ ■■■ utils/
  ■ ■ ■■■ word_placer.dart # 8-direction word placement
  ■ ■ ■■■ grid_generator.dart
  ■ ■ ■■■ word_validator.dart
  ■ ■ ■■■ grid_rotator.dart
  ■ ■ ■■■ widgets/
  ■ ■ ■■■ gradient_button.dart
  ■ ■ ■■■ coin_display.dart
  ■ ■ ■■■ services/
  ■ ■ ■■■ audio_service.dart
  ■ ■ ■■■ ad_service.dart
  ■ ■ ■■■ analytics_service.dart
  ■ ■
  ■ ■■■ features/
  ■ ■ ■■■ splash/ → presentation/screens/splash_screen.dart
  ■ ■ ■■■ home/ → presentation/screens/home_screen.dart
  ■ ■ ■ presentation/widgets/ (world_map, destination_card)
  ■ ■ ■■■ level_select/
  ■ ■ ■ ■■■ data/models/ + repositories/
  ■ ■ ■ ■■■ domain/entities/ + usecases/
  ■ ■ ■ ■■■ presentation/cubit/ + screens/ + widgets/
  ■ ■ ■■■ game/
  ■ ■ ■ ■■■ data/models/ (grid_cell, word, level)
  ■ ■ ■ ■■■ domain/entities/ + repositories/ + usecases/
  ■ ■ ■ ■■■ presentation/
  ■ ■ ■ ■■■ bloc/ (game_bloc, game_event, game_state)
  ■ ■ ■ ■■■ cubit/ (coin_cubit, timer_cubit, hint_cubit)
  ■ ■ ■ ■■■ screens/ (game_screen.dart)
  ■ ■ ■ ■■■ widgets/ (letter_grid, word_list_panel,
  ■ ■ ■ game_toolbar, level_complete_overlay)
  ■ ■ ■■■ shop/ → presentation/cubit/ + screens/ + widgets/
  ■ ■ ■■■ profile/ → presentation/screens/
  ■ ■ ■■■ settings/ → presentation/screens/
  ■ ■
  ■ ■■■ data/
  ■ ■■■ local/ (database.dart, tables.dart, prefs.dart)
  ■ ■■■ words/ (word_categories.dart)
  ■
  ■■■ assets/
  ■ ■■■ data/ (levels_pack_1.json, levels_pack_2.json, words_dictionary.json)
  ■ ■■■ images/ (splash/, themes/)
  ■ ■■■ animations/ (confetti.json, star_burst.json, word_found.json)
  ■ ■■■ audio/ (word_found.mp3, level_complete.mp3, wrong.mp3, bg_music.mp3)
  ■
```

```
■■■ test/
■ ■■■ unit/ (word_placer_test, grid_rotator_test, word_validator_test)
■ ■■■ widget/ (letter_grid_test, game_bloc_test)
■ ■■■ integration/ (game_flow_test)
■
■■■ pubspec.yaml
```

## 8. PUBSPEC.YAML — ALL DEPENDENCIES

```
# pubspec.yaml
name: word_search_game
description: Word Search Journey Flutter Clone
version: 1.0.0+1

environment:
  sdk: '>=3.4.0 <4.0.0'
  flutter: ">=3.22.0"

dependencies:
  flutter:
    sdk: flutter

  # State Management
  flutter_bloc: ^8.1.6
  equatable: ^2.0.5
  bloc: ^8.1.4

  # Dependency Injection
  get_it: ^7.7.0
  injectable: ^2.4.0

  # Navigation
  go_router: ^14.2.7

  # Database & Storage
  drift: ^2.18.0
  sqlite3_flutter_libs: ^0.5.24
  path_provider: ^2.1.3
  shared_preferences: ^2.3.1

  # Ads & Monetization
  google_mobile_ads: ^5.1.0
  in_app_purchase: ^3.2.0

  # UI & Animation
  lottie: ^3.1.2
  animated_text_kit: ^4.2.2
  shimmer: ^3.0.0
  confetti: ^0.7.0
  flutter_animate: ^4.5.0
  cached_network_image: ^3.3.1

  # Audio
  audioplayers: ^6.0.0

  # Firebase
  firebase_core: ^3.5.0
  firebase_analytics: ^11.3.0
  firebase_crashlytics: ^4.1.0

  # Utilities
  intl: ^0.19.0
  dartz: ^0.10.1
  google_fonts: ^6.2.1

dev_dependencies:
  build_runner: ^2.4.11
  drift_dev: ^2.18.0
  injectable_generator: ^2.6.1
  freezed: ^2.5.5
  mocktail: ^1.0.4

flutter:
  uses-material-design: true
  assets:
```

```
- assets/data/levels_pack_1.json
- assets/data/levels_pack_2.json
- assets/data/words_dictionary.json
- assets/images/
- assets/animations/
- assets/audio/
```

## 9. CORE GAME ALGORITHMS — FULL CODE

### 9.1 Word Placement Algorithm (8 Directions)

```
// lib/core/utils/word_placer.dart
import 'dart:math';

enum Direction { right, left, down, up, downRight, downLeft, upRight, upLeft }

class WordPlacement {
  final String word;
  final int startRow, startCol;
  final Direction direction;
  final List<({int row, int col})> cells;
}

class WordPlacer {
  static const Map> _dirs = {
    Direction.right: [0, 1],
    Direction.left: [0, -1],
    Direction.down: [1, 0],
    Direction.up: [-1, 0],
    Direction.downRight: [1, 1],
    Direction.downLeft: [1, -1],
    Direction.upRight: [-1, 1],
    Direction.upLeft: [-1, -1],
  };

  final int gridSize;
  final Random _random;

  WordPlacer({required this.gridSize, int? seed}) : _random = Random(seed);

  PlacementResult placeWords(List words) {
    final grid = List.generate(gridSize, (_) => List.filled(gridSize, ''));
    final placements = [];
    final sorted = [...words]..sort((a, b) => b.length.compareTo(a.length));

    for (final word in sorted) {
      final placement = _tryPlaceWord(grid, word.toUpperCase());
      if (placement == null) return placeWords(words); // restart
      _applyPlacement(grid, placement);
      placements.add(placement);
    }

    _fillRandom(grid);
    return PlacementResult(grid: grid, placements: placements);
  }

  WordPlacement? _tryPlaceWord(List> grid, String word) {
    final dirs = Direction.values.toList()..shuffle(_random);
    for (int attempt = 0; attempt < 100; attempt++) {
      final dir = dirs[attempt % dirs.length];
      final delta = _dirs[dir]!;
      final row = _random.nextInt(gridSize);
      final col = _random.nextInt(gridSize);
      if (_canPlace(grid, word, row, col, delta)) {
        final cells = List.generate(word.length, (i) =>
          (row: row + delta[0]*i, col: col + delta[1]*i));
        return WordPlacement(word: word, startRow: row, startCol: col,
          direction: dir, cells: cells);
      }
    }
    return null;
  }

  bool _canPlace(List> grid, String word, int row, int col, Direction dir) {
    for (int i = 0; i < word.length; i++) {
      int r = row + dir[0]*i;
      int c = col + dir[1]*i;
      if (r < 0 || r >= gridSize || c < 0 || c >= gridSize || grid[r][c] != '') return false;
    }
    return true;
  }

  void _fillRandom(List> grid) {
    for (int r = 0; r < gridSize; r++) {
      for (int c = 0; c < gridSize; c++) {
        if (grid[r][c] == '') {
          final word = _random.nextString(4).toUpperCase();
          final placement = _tryPlaceWord(grid, word);
          if (placement != null) _applyPlacement(grid, placement);
        }
      }
    }
  }

  void _applyPlacement(List> grid, WordPlacement placement) {
    for ({int row, int col} cell in placement.cells) {
      grid[row][col] = placement.word[cell.col - placement.startCol];
    }
  }
}
```

```

}

bool _canPlace(List<String> grid, String word, int r, int c, List d) {
  for (int i = 0; i < word.length; i++) {
    final nr = r + d[0] * i;
    final nc = c + d[1] * i;
    if (nr < 0 || nr >= gridSize || nc < 0 || nc >= gridSize) return false;
    if (grid[nr][nc].isEmpty && grid[nr][nc] != word[i]) return false;
  }
  return true;
}

void _fillRandom(List<String> grid) {
  const letters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
  for (int r = 0; r < gridSize; r++)
    for (int c = 0; c < gridSize; c++)
      if (grid[r][c].isEmpty) grid[r][c] = letters[_random.nextInt(26)];
}

```

## 9.2 Grid Rotation Algorithm

```

// lib/core/utils/grid_rotator.dart
class GridRotator {
  /// Rotate 90 degrees clockwise
  static List<String> rotate90CW(List<String> grid) {
    final n = grid.length;
    final rotated = List.generate(n, (_) => List.filled(n, grid[0][0]));
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        rotated[j][n - 1 - i] = grid[i][j];
    return rotated;
  }

  static List rotatePlacements(
    List placements, int gridSize) {
    return placements.map((p) {
      final newCells = p.cells.map((cell) =>
        (row: cell.col, col: gridSize - 1 - cell.row)).toList();
      return WordPlacement(
        word: p.word, startRow: newCells.first.row, startCol: newCells.first.col,
        direction: _rotateDirection(p.direction), cells: newCells,
      );
    }).toList();
  }

  static Direction _rotateDirection(Direction d) => switch(d) {
    Direction.right => Direction.down,
    Direction.down => Direction.left,
    Direction.left => Direction.up,
    Direction.up => Direction.right,
    Direction.downRight => Direction.downLeft,
    Direction.downLeft => Direction.upLeft,
    Direction.upLeft => Direction.upRight,
    Direction.upRight => Direction.downRight,
  };
}

```

## 9.3 Swipe/Gesture Detection (CustomPainter Grid)

```
// lib/features/game/presentation/widgets/letter_grid.dart
class LetterGrid extends StatefulWidget {
  final List<String> grid;
  final List<int> selectedIndices;
  final List<int> foundIndices;
  final Function(int row, int col) onDragStart;
  final Function(int row, int col) onDragUpdate;
  final VoidCallback onDragEnd;
}

class _LetterGridState extends State {
  late double cellSize;

  ({int row, int col})? _getCellFromOffset(Offset local) {
    final col = (local.dx / cellSize).floor();
    final row = (local.dy / cellSize).floor();
    final n = widget.grid.length;
    if (row < 0 || row >= n || col < 0 || col >= n) return null;
    return (row: row, col: col);
  }

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(builder: (ctx, constraints) {
      cellSize = constraints.maxWidth / widget.grid.length;
      return GestureDetector(
        onPanStart: (d) {
          final cell = _getCellFromOffset(d.localPosition);
          if (cell != null) widget.onDragStart(cell.row, cell.col);
        },
        onPanUpdate: (d) {
          final cell = _getCellFromOffset(d.localPosition);
          if (cell != null) widget.onDragUpdate(cell.row, cell.col);
        },
        onPanEnd: (_) => widget.onDragEnd(),
        child: CustomPaint(
          painter: GridPainter(
            grid: widget.grid, cellSize: cellSize,
            selectedIndices: widget.selectedIndices,
            foundIndices: widget.foundIndices,
          ),
          size: Size.square(constraints.maxWidth),
        ),
      );
    });
  }
}

class GridPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final n = grid.length;
    for (int r = 0; r < n; r++) {
      for (int c = 0; c < n; c++) {
        final idx = r * n + c;
        final rect = Rect.fromLTWH(c*cellSize, r*cellSize, cellSize, cellSize);
        Color bg = AppColors.cellDefault;
        if (foundIndices.contains(idx)) bg = AppColors.cellFound;
        else if (selectedIndices.contains(idx)) bg = AppColors.cellSelected;
        canvas.drawRect(rect, Paint()..color = bg);
        // Draw border and letter...
      }
    }
  }
}
```

```
}  
@override bool shouldRepaint(GridPainter old) =>  
old.selectedIndices != selectedIndices || old.foundIndices != foundIndices;  
}
```

## 10. ALL SCREENS — FULL FLUTTER CODE

---

### 10.1 main.dart

```
// lib/main.dart
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'injection.dart';
import 'app.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  configureDependencies();
  SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp]);
  SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(
    statusBarColor: Colors.transparent,
    statusBarIconBrightness: Brightness.light,
  ));
  runApp(const WordSearchApp());
}
```

### 10.2 app.dart — GoRouter

```
// lib/app.dart
class WordSearchApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'Word Search Journey',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: AppColors.primaryBlue),
        textTheme: GoogleFonts.poppinsTextTheme(),
        useMaterial3: true,
        scaffoldBackgroundColor: AppColors.lightBlueBg,
      ),
      routerConfig: _router,
    );
  }
}

final _router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(path: '/', builder: (ctx, state) => const SplashScreen()),
    GoRoute(path: '/home', builder: (ctx, state) => const HomeScreen()),
    GoRoute(path: '/levels', builder: (ctx, state) {
      final themeId = int.parse(state.extra as String? ?? '1');
      return LevelSelectScreen(themeId: themeId);
    }),
    GoRoute(path: '/game', builder: (ctx, state) {
      final levelId = int.parse(state.extra as String? ?? '101');
      return GameScreen(levelId: levelId);
    }),
    GoRoute(path: '/shop', builder: (ctx, state) => const ShopScreen()),
  ],
);
```

```
GoRoute(path: '/profile', builder: (ctx, state) => const ProfileScreen()),
GoRoute(path: '/settings', builder: (ctx, state) => const SettingsScreen()),
],
);
```

## 10.3 Splash Screen

```
// lib/features/splash/presentation/screens/splash_screen.dart
class SplashScreen extends StatefulWidget { ... }

class _SplashScreenState extends State
with SingleTickerProviderStateMixin {
  late AnimationController _ctrl;
  late Animation _fadeAnim;

  @override
  void initState() {
    super.initState();
    _ctrl = AnimationController(vsync: this,
      duration: const Duration(milliseconds: 2500));
    _fadeAnim = Tween(begin: 0.0, end: 1.0).animate(
      CurvedAnimation(parent: _ctrl, curve: Curves.easeIn));
    _ctrl.forward();
    Future.delayed(const Duration(milliseconds: 3000), () {
      if (mounted) context.go('/home');
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Stack(
        fit: StackFit.expand,
        children: [
          Image.asset('assets/images/splash/travel_bg.jpg', fit: BoxFit.cover),
          Container(color: Colors.black.withOpacity(0.45)),
          FadeTransition(
            opacity: _fadeAnim,
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Lottie.asset('assets/animations/word_logo.json',
                  width: 200, height: 200, repeat: false),
                const SizedBox(height: 24),
                Text('WORD SEARCH JOURNEY',
                  style: TextStyle(fontFamily: 'Montserrat', fontSize: 28,
                    fontWeight: FontWeight.w900, color: Colors.white)),
                Text('Travel the world with words',
                  style: TextStyle(fontSize: 14, color: Colors.white70)),
                const SizedBox(height: 48),
                SizedBox(width: 200,
                  child: LinearProgressIndicator(
                    backgroundColor: Colors.white24,
                    valueColor: AlwaysStoppedAnimation(AppColors.primaryBlue),
                  )),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

```

}
}

```

## 10.4 Game Screen (Core)

```

// lib/features/game/presentation/screens/game_screen.dart
class GameScreen extends StatefulWidget {
  final int levelId;
  const GameScreen({super.key, required this.levelId});
}

class _GameScreenState extends State {
  late ConfettiController _confettiCtrl;

  @override
  void initState() {
    super.initState();
    _confettiCtrl = ConfettiController(duration: const Duration(seconds: 3));
    context.read().add(LoadLevel(widget.levelId));
  }

  @override
  Widget build(BuildContext context) {
    return BlocListener(
      listener: (ctx, state) {
        if (state is GameCompleted) {
          _confettiCtrl.play();
          showDialog(context: ctx, barrierDismissible: false,
            builder: (_) => LevelCompleteOverlay(
              stars: state.stars, coinsEarned: state.coinsEarned,
              time: state.time, hintsUsed: state.hintsUsed));
        }
      },
      child: Scaffold(
        backgroundColor: AppColors.lightBlueBg,
        body: SafeArea(
          child: Stack(
            children: [
              Column(children: [
                _buildTopBar(), // ← Back | Theme | Stars | Timer | Settings
                _buildCoinHintRow(), // ← Coins | Hints | Pause
                _buildWordListPanel(), // ← Horizontal scrollable word chips
                _buildGrid(), // ← CustomPainter letter grid
                _buildToolbar(), // ← Rotate | Hint | Reveal | Shuffle
              ]),
              Align(alignment: Alignment.topCenter,
                child: ConfettiWidget(
                  confettiController: _confettiCtrl,
                  blastDirectionality: BlastDirectionality.explosive,
                  colors: [AppColors.gold, AppColors.primaryBlue, AppColors.success],
                )),
            ],
          ),
        ),
      );
  }

  Widget _buildGrid() {
    return Expanded(child: Padding(
      padding: const EdgeInsets.all(8),

```

```
child: BlocBuilder(builder: (ctx, state) {
  if (state is GameInProgress) {
    return LetterGrid(
      grid: state.grid,
      selectedIndices: state.selectedCells.map((c) =>
        c.row * state.grid.length + c.col).toList(),
      foundIndices: state.foundCells,
      onDragStart: (r, c) =>
        ctx.read().add(CellDragStarted(row: r, col: c)),
      onDragUpdate: (r, c) =>
        ctx.read().add(CellDragUpdated(row: r, col: c)),
      onDragEnd: () =>
        ctx.read().add(const CellDragEnded()),
    );
  }
  return const Center(child: CircularProgressIndicator());
}),
));
}
```

## 11. BLOC STATE MANAGEMENT — FULL CODE

### 11.1 game\_event.dart

```
// lib/features/game/presentation/bloc/game_event.dart
abstract class GameEvent extends Equatable {
  const GameEvent();
}

class LoadLevel extends GameEvent { final int levelId; }
class CellDragStarted extends GameEvent { final int row, col; }
class CellDragUpdated extends GameEvent { final int row, col; }
class CellDragEnded extends GameEvent { const CellDragEnded(); }
class HintRequested extends GameEvent { const HintRequested(); }
class RevealRequested extends GameEvent { const RevealRequested(); }
class BoardRotated extends GameEvent { const BoardRotated(); }
class ShuffleRequested extends GameEvent { const ShuffleRequested(); }
class GamePaused extends GameEvent { const GamePaused(); }
class GameResumed extends GameEvent { const GameResumed(); }
class ExtraTimeAdded extends GameEvent { const ExtraTimeAdded(); }
```

### 11.2 game\_state.dart

```
// lib/features/game/presentation/bloc/game_state.dart
enum SelectionState { idle, selecting, correct, wrong }
enum DifficultyLevel { easy, medium, hard, pro }

abstract class GameState extends Equatable {}

class GameInitial extends GameState {}
class GameLoading extends GameState {}

class GameInProgress extends GameState {
  final List<String> grid;
  final List<String> wordsToFind, foundWords;
  final List<Cell> selectedCells;
  final List<Cell> foundCells; // flat cell indices
  final int coins, hintsLeft, timeLimit, levelId;
  final Duration elapsed;
  final bool isPaused;
  final DifficultyLevel difficulty;
  final SelectionState selectionState;
  final String levelTheme;
  final int currentStars;

  bool get allWordsFound => foundWords.length == wordsToFind.length;
  int get remaining => timeLimit > 0 ? (timeLimit - elapsed.inSeconds).clamp(0, 9999) : 0;

  GameInProgress copyWith({ ... }) => GameInProgress( ... );
}

class GameCompleted extends GameState {
  final int stars, coinsEarned, hintsUsed;
  final Duration time;
}

class GameError extends GameState {
  final String message;
}
```

## 11.3 game\_bloc.dart (Core Logic)

```
// lib/features/game/presentation/bloc/game_bloc.dart
class GameBloc extends Bloc {
  Timer? _timer;
  List _placements = [];

  GameBloc({ ... }) : super(const GameInitial()) {
    on(_onLoadLevel);
    on(_onDragStart);
    on(_onDragUpdate);
    on(_onDragEnd);
    on(_onHint);
    on(_onReveal);
    on(_onRotate);
    on(_onPause);
    on(_onResume);
  }

  Future _onLoadLevel(LoadLevel event, Emitter emit) async {
    emit(const GameLoading());
    final levelResult = await _loadLevel(event.levelId);
    levelResult.fold(
      (failure) => emit(GameError(failure.message)),
      (level) {
        final result = WordPlacer(gridSize: level.gridSize).placeWords(level.words);
        _placements = result.placements;
        emit(GameInProgress( grid: _buildGridModels(result.grid), ... ));
        _startTimer();
      },
    );
  }

  Future _onDragEnd(CellDragEnded e, Emitter emit) async {
    final s = state as GameInProgress?;
    if (s == null) return;
    final word = s.selectedCells.map((c) => c.letter).join();
    final placement = _placements.firstWhere(
      (p) => p.word == word || p.word == word.split('').reversed.join(''),
      orElse: () => emptyPlacement);
    if (placement.word.isNotEmpty && !s.foundWords.any((w) => w.text == placement.word)) {
      // CORRECT WORD
      _audio.playWordFound();
      final newState = s.copyWith(
        foundWords: [...s.foundWords, WordModel(text: placement.word, isFound: true)],
        foundCells: [...s.foundCells, ...placement.cells.map((c) => c.row * n + c.col)],
        selectedCells: [], selectionState: SelectionState.correct,
      );
      if (newState.allWordsFound) _completeGame(newState);
      else emit(newState);
    } else {
      // WRONG — flash red then reset
      emit(s.copyWith(selectionState: SelectionState.wrong));
      await Future.delayed(const Duration(milliseconds: 500));
      emit(s.copyWith(selectedCells: [], selectionState: SelectionState.idle));
    }
  }

  void _onRotate(BoardRotated e, Emitter emit) {
    final s = state as GameInProgress?;
    if (s == null) return;
    _placements = GridRotator.rotatePlacements(_placements, s.grid.length);
  }
}
```

```
emit(s.copyWith(grid: GridRotator.rotate90CW(s.grid), selectedCells: []));  
}  
  
int _calculateStars(GameInProgress s) {  
  final ratio = s.elapsed.inSeconds / (s.timeLimit > 0 ? s.timeLimit : 180);  
  if (ratio < 0.5) return 3;  
  if (ratio < 0.8) return 2;  
  return 1;  
}  
}
```

## 12. DATA LAYER — DATABASE & MODELS

### 12.1 Drift SQLite Database

```
// lib/data/local/database.dart
import 'package:drift/drift.dart';
import 'package:drift/native.dart';

@DriftDatabase(tables: [LevelsTable, UserProgressTable, WordsTable, DailyTable])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(_openConnection());

  @override int get schemaVersion => 1;

  Future getProgress(int levelId) =>
    (select(userProgressTable)..where((t) => t.levelId.equals(levelId)))
    .getSingleOrNull();

  Future saveProgress(int levelId, int stars, int timeSeconds) =>
    into(userProgressTable).insertOnConflictUpdate(
      UserProgressTableCompanion(
        levelId: Value(levelId), stars: Value(stars),
        timeSeconds: Value(timeSeconds),
        completedAt: Value(DateTime.now()),
      ));

  Future getCoins() async {
    final row = await customSelect('SELECT value FROM kv WHERE key = ?',
      variables: [const Variable('coins')]).getSingleOrNull();
    return int.tryParse(row?.data['value'] ?? '250') ?? 250;
  }
}
```

### 12.2 Level JSON Structure

```
{
  "version": "1.0",
  "categories": [
    {
      "id": 1, "name": "Paris Adventure",
      "theme": "travel_europe", "backgroundImage": "paris_bg.jpg",
      "levels": [
        {
          "id": 101, "difficultyIndex": 0, "gridSize": 8,
          "timeLimit": 180, "hintsAllowed": 3, "coinsReward": 30,
          "words": ["EIFFEL", "PARIS", "LOUVRE", "CAFE", "METRO", "BERET", "WINE"]
        },
        {
          "id": 102, "difficultyIndex": 1, "gridSize": 10,
          "timeLimit": 240, "hintsAllowed": 2, "coinsReward": 50,
          "words": ["SEINE", "BAGUETTE", "VERSAILLES", "MONTMARTRE", "NAPOLEON"]
        },
        {
          "id": 103, "difficultyIndex": 2, "gridSize": 12,
          "timeLimit": 300, "hintsAllowed": 2, "coinsReward": 70,
          "words": ["CHAMPS", "ELYSEES", "MUSEE", "ORSAY", "SACRE", "BASTILLE"]
        },
        {
          "id": 104, "difficultyIndex": 3, "gridSize": 14,

```

```
"timeLimit": 360, "hintsAllowed": 1, "coinsReward": 100,  
"words": ["CATACOMBS", "POMPIDOU", "PALAISROYAL", "GRANDPALAIS"]  
}  
},  
{  
  "id": 2, "name": "Tokyo Journey",  
  "levels": [  
    { "id": 201, "gridSize": 8, "difficultyIndex": 0,  
      "words": ["TOKYO", "SUSHI", "RAMEN", "NINJA", "SAKURA", "FUJI"] }  
  ]  
}  
]  
}
```

## 13. MONETIZATION — ADMOB + IAP CODE

### 13.1 AdMob Service

```
// lib/core/services/ad_service.dart
@singleton
class AdService {
  InterstitialAd? _interstitial;
  RewardedAd? _rewarded;
  int _levelCount = 0;

  // Test IDs (replace with production IDs for release)
  static String get _interstitialId => Platform.isAndroid
    ? 'ca-app-pub-3940256099942544/1033173712'
    : 'ca-app-pub-3940256099942544/4411468910';

  static String get _rewardedId => Platform.isAndroid
    ? 'ca-app-pub-3940256099942544/5224354917'
    : 'ca-app-pub-3940256099942544/1712485313';

  Future initialize() async {
    await MobileAds.instance.initialize();
    loadInterstitial();
    loadRewarded();
  }

  // Show interstitial every 3rd level completion only
  void onLevelComplete() {
    _levelCount++;
    if (_levelCount % 3 == 0) showInterstitial();
  }

  Future showRewardedAd({
    required Function(int coins) onReward,
    required VoidCallback onFail,
  }) async {
    if (_rewarded == null) { onFail(); return false; }
    await _rewarded!.show(
      onUserEarnedReward: (_, reward) => onReward(50),
    );
    return true;
  }
}
```

### 13.2 IAP Products

```
// lib/features/shop/presentation/cubit/shop_cubit.dart
const kProducts = {
  'coins_500' : 0.99,
  'coins_1500' : 2.49, // Best Value
  'coins_5000' : 7.99,
  'remove_ads' : 1.99, // One-time purchase
  'vip_monthly' : 0.99, // Subscription
  'starter_pack' : 0.49, // New user only
};

class ShopCubit extends Cubit {
  final InAppPurchase _iap = InAppPurchase.instance;

  Future _init() async {
    final available = await _iap.isAvailable();
  }
}
```

```
if (!available) { emit(const ShopUnavailable()); return; }
final response = await _iap.queryProductDetails(kProducts.keys.toSet());
emit(ShopLoaded(products: response.productDetails));
_iap.purchaseStream.listen(_handlePurchase);
}

void _handlePurchase(List purchases) {
  for (final p in purchases) {
    if (p.status == PurchaseStatus.purchased) {
      _deliverProduct(p.productID);
      _iap.completePurchase(p);
    }
  }
}
```

### 13.3 Revenue Strategy

| Ad Format      | Placement / Trigger             | eCPM Range | User Impact                 |
|----------------|---------------------------------|------------|-----------------------------|
| Interstitial   | After every 3rd level           | \$2–\$8    | Skippable after 5s — Medium |
| Rewarded Video | Watch for +50 coins / 2x reward | \$8–\$25   | Player-initiated — Very Low |
| Banner         | Bottom of Level Select screen   | \$0.3–\$1  | Always visible — Very Low   |
| App Open       | 1st open per session            | \$2–\$5    | Infrequent — Low            |

## 14. ADDITIONAL FEATURES CODE

### 14.1 Daily Challenge System

```
// lib/core/utils/daily_challenge_service.dart
class DailyChallengeService {
  // All players on same day get same puzzle (same seed = same grid)
  static int getDailySeed() {
    final now = DateTime.now();
    return now.year * 10000 + now.month * 100 + now.day;
  }

  static bool hasCompletedToday(SharedPreferences prefs) {
    final last = prefs.getString('last_daily_date') ?? '';
    return last == DateTime.now().toIso8601String().substring(0, 10);
  }

  static List getDailyWords() {
    final r = Random(getDailySeed());
    const allWords = ['OCEAN', 'BEACH', 'CORAL', 'WAVE', 'SHELL', 'FISH',
      'SAND', 'TIDE', 'REEF', 'CRAB', 'SAIL', 'PORT'];
    return List.generate(7, (_) => allWords[r.nextInt(allWords.length)])
      .toSet().take(6).toList();
  }
}
```

### 14.2 Achievement System

```
// lib/features/profile/domain/entities/achievement.dart
const kAchievements = [
  Achievement(id: 'first_word', title: 'Word Novice',
    description: 'Find your first word'),
  Achievement(id: 'speed_demon', title: 'Speed Demon',
    description: 'Complete level under 60 seconds'),
  Achievement(id: 'hint_free', title: 'Hint-Free Hero',
    description: 'Complete 10 levels without hints'),
  Achievement(id: 'world_explore', title: 'World Explorer',
    description: 'Complete all levels in one theme'),
  Achievement(id: 'daily_7', title: 'Daily Devotee',
    description: '7-day login streak'),
  Achievement(id: 'vocab_master', title: 'Vocabulary Master',
    description: 'Find 1000 total words'),
];
```

### 14.3 Audio Service

```
// lib/core/services/audio_service.dart
@singleton
class AudioService {
  final AudioPlayer _sfxPlayer = AudioPlayer();
  final AudioPlayer _bgPlayer = AudioPlayer();

  bool _soundEnabled = true;
  bool _musicEnabled = true;

  Future playWordFound() async => _play('audio/word_found.mp3');
  Future playLevelComplete() async => _play('audio/level_complete.mp3');
```

```
Future playWrong() async => _play('audio/wrong.mp3');

Future startBackgroundMusic() async {
  if (!_musicEnabled) return;
  await _bgPlayer.setReleaseMode(ReleaseMode.loop);
  await _bgPlayer.play(AssetSource('audio/bg_music.mp3'), volume: 0.4);
}

void toggleSound() { _soundEnabled = !_soundEnabled; }

void toggleMusic() {
  _musicEnabled = !_musicEnabled;
  _musicEnabled ? startBackgroundMusic() : _bgPlayer.pause();
}
}
```

## 15. DEVELOPMENT ROADMAP & LAUNCH CHECKLIST

### 15.1 12-Week Development Plan

| Phase   | Timeline | Deliverables                                                                                                         | Priority   |
|---------|----------|----------------------------------------------------------------------------------------------------------------------|------------|
| Phase 1 | Week 1–2 | Project setup, pubspec, folder structure, GetIt DI, GoRouter, BLoC setup, app_colors, basic navigation               | ■ Critical |
| Phase 2 | Week 3–5 | WordPlacer algorithm, GridRotator, LetterGrid widget, swipe detection, GameBloc (all events/states), word validation | ■ Critical |
| Phase 3 | Week 6–7 | 100 levels JSON data, 5 themes, word categories, SplashScreen, HomeScreen, LevelSelectScreen                         | ■ High     |
| Phase 4 | Week 8–9 | Lottie animations, confetti, shimmer effects, all screen transitions, LevelCompleteOverlay, ShopScreen               | ■ High     |
| Phase 5 | Week 10  | AdMob integration (all ad types), IAP setup (6 products), CoinCubit wired, rewarded video flow                       | ■ High     |
| Phase 6 | Week 11  | Unit tests (WordPlacer, GridRotator), widget tests (LetterGrid), BLoC tests, Firebase Crashlytics                    | ■ Critical |
| Phase 7 | Week 12  | Play Store listing, ASO optimization, signed AAB, screenshots, privacy policy, launch!                               | ■ Critical |

### 15.2 Play Store Launch Checklist

| Item              | Specification                                                          |
|-------------------|------------------------------------------------------------------------|
| App Icon          | 512x512px PNG + adaptive icon (foreground + background layers)         |
| Feature Graphic   | 1024x500px PNG (Play Store banner)                                     |
| Screenshots       | 8–12 screenshots: phone (1080x1920) + tablet (1600x2560)               |
| Short Description | 80 chars max: "Explore the world through words — offline puzzle game!" |
| Full Description  | 4000 chars — keywords: word search, puzzle, word find, offline         |
| ASO Keywords      | word search, word puzzle, word find, brain game, offline puzzle        |
| Content Rating    | Questionnaire completed — should be 3+ (Everyone)                      |
| Privacy Policy    | URL required (for ads). Use Termly or PrivacyPolicies.com              |
| Signed AAB        | Release keystore stored securely (NEVER commit to Git)                 |
| Firebase          | Crashlytics + Analytics enabled — test crash reporting before launch   |
| Ad IDs            | Replace all test Ad Unit IDs with production IDs from AdMob dashboard  |
| IAP Products      | All 6 products created & approved in Google Play Console               |
| Offline Test      | Test all 100 levels work completely offline (airplane mode)            |
| Performance       | Test on low-end device (2GB RAM, Android 5.0) — target 60fps           |

### 15.3 Revenue Projection

**■ Revenue Formula for Word Search Games**

Monthly Revenue = (DAU × Ad ARPDAU × 30) + (MAU × IAP Conversion% × Avg IAP)

Ad ARPDAU (Word games): \$0.02 – \$0.08 per user per day

IAP Conversion Rate: 2–5% of MAU | Average IAP Value: \$1.50 – \$3.00

EXAMPLE at 10,000 DAU:

- Ads:  $10,000 \times \$0.04 \times 30 \text{ days} = \$12,000/\text{month}$
- IAP:  $30,000 \text{ MAU} \times 3\% \times \$2.00 = \$1,800/\text{month}$
- TOTAL: ~\$13,800/month at 10K DAU

Break-even: ~Month 4–6 after UA investment recovery

Rotate Board = FREE always → highest user retention driver

**■ Document Complete — Word Search Journey Flutter Clone Blueprint**

*Full Technical Blueprint for Cursor AI | Prepared May 2026*

Clean Architecture • BLoC Pattern • 8-Direction Grid • AdMob + IAP • Offline-First